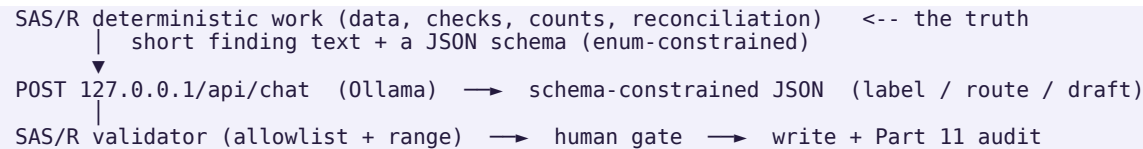


LOCALMIND — a SAS/R helper library for a LOCAL, OFFLINE small language model

What it is. A small, deterministic library that lets SAS or R call an **on-device small language model (Ollama / llama.cpp on `127.0.0.1`, fully offline) for the language layer only — classify, extract, route, draft — and nothing else.** SAS/R owns every number and check; the model never produces a count, and a human approves anything consequential. **It is the on-device-SLM analogue of the SHEETLINK and TRIALMON companions.**



Files

File	Role
slm_macros.sas	The SAS library — every macro is %slm_* (PROC HTTP + JSON engine).
slm_companion.R	The R equivalent (httr2 + jsonlite).
slm_config.sas	The pinned model + the controlled vocabularies (allowlists).
trriage_driver.sas	The worked example as runnable SAS (QC-finding triage).

The five guarantees

- Offline — nothing leaves the box.** %slm_init / slm_init() **refuse** any non-loopback endpoint (127.0.0.1 / localhost / [::1] only) and probe the local server's liveness — the in-code "no egress" boundary (the network-destination analogue of SHEETLINK's ops-only allowlist). Run the box with the model service bound to loopback and the cable out, and egress is physically impossible.
- Schema-constrained output.** Every call sends a **full JSON Schema** as the grammar constraint (Ollama format, llama.cpp json_schema/GBNF), so the model can only emit allowed tokens — enum fields are pre-narrowed to your controlled vocabulary at *sample* time. JSON is **guaranteed parseable**, not best-effort.
- Validated against an allowlist — the model is untrusted text.** %slm_validate / slm_validate() parse the JSON and check **every controlled field** against a hard-coded allowlist (UPCASE) and numeric fields against a range; an off-list value (a hallucinated category) or a bad number is **rejected and fails loud — never coerced**. Grammar-constrain at the server *and* allowlist-validate at the client: belt and suspenders.
- Pinned & frozen for reproducibility.** The model **tag + quantization + sha256 digest**, temperature 0, and a **fixed seed** are pinned and stamped onto every result, so a run is a re-runnable artifact for GAMP-5 / 21 CFR Part 11. (Honest caveat: greedy decoding is reproducible on the **same box/build** — pin the hardware, not just the params.)

5. **Human-gated — the model owns no record.** Drafts/labels are advisory; SAS/R writes only after a person approves, and emits the Part 11 audit line (prompt, model digest, draft, reviewer, time). **The model never produces a reported number.**

Quick start (SAS)

```
%include "slm_config.sas"; /* pins CFG_MODEL + the allowlists */
%include "slm_macros.sas";
%slm_init(model=&CFG_MODEL, base=&CFG_BASE, seed=&CFG_SEED, backup=&CFG_BACKUP);

/* the finding text comes from SAS-owned validated work; set it via SYMPUTX (safe for commas) */
data _null_; call symputx('_SLM_USR',
  'AESTDTC is after AEENDTC for 3 records in domain AE.', 'L'); run;

%slm_classify(sys=%nrstr(You route ONE QC finding. Use only the schema. Never invent a number.),
  field=owner, allow=&CFG_OWNERS, out=routed);
/* `routed` now has: owner (validated to CFG_OWNERS), confidence, rationale, + provenance cols */
```

Quick start (R)

```
source("slm_companion.R")
con <- slm_init("qwen2.5:7b-instruct-q8_0") # loopback-guarded, asserts the model is pulled
r <- slm_classify(con,
  sys = "You route ONE QC finding. Use only the schema. Never invent a number.",
  usr = "AESTDTC is after AEENDTC for 3 records in domain AE.",
  field = "owner", allow = c("DM", "PROGRAMMING", "PK", "MEDICAL CODING", "SITE", "BIOSTAT"))
```

Macro / function reference

SAS	R	Does
%slm_init	slm_init	Pin model; assert loopback/offline ; assert the model is pulled; capture the digest.
%slm_chat	(inside slm_classify)	One /api/chat call: stream: false, temp 0 + seed, schema-constrained, double-parse the result.
%slm_classify	slm_classify	Build a one-enum schema from an allowlist, call, validate . The flagship.
%slm_validate	slm_validate	The trust boundary: allowlist + range check; reject + fail loud.
%slm_assert	(stop())	Fail loud + notify backup.

Things that will bite you (from the field)

- **Streaming is ON by default.** Always send stream: false, or PROC HTTP / a single resp_body_json sees many NDJSON objects and breaks.
- **Double-parse.** The schema result comes back as a *JSON string*** inside message.content; parse the envelope, then parse content again.
- **The model must be pulled** (ollama pull <tag>); %slm_init asserts it via /api/tags.
- **Context limit truncates silently.** Set num_ctx and keep num_predict generous (a cut closing brace = a parse failure); **chunk** long inputs — SAS/R should never hand a small model a long document.

- **Quantization is part of the model's identity.** A q4 model drifts more than q8; pin the exact quant **and** its sha256 — don't compare a q4 run to a q8 baseline.
- **JSON-engine table names vary by SAS version.** %slm_chat reads the inner object from _p.root and the tags list from _slmj.models / the digest from _slmj.details; if your version names them ALLDATA / model_info, run one call and PROC DATASETS lib=_p; to confirm, then adjust. Same one-time check as any JSON-libname code.
- **Determinism is per-box.** temp 0 + seed is reproducible on one machine/build; GPU FP non-associativity and a backend version bump can shift a token. Pin the hardware and run one slot (OLLAMA_NUM_PARALLEL=1). Validate against a sandbox before production.

The honest boundary

The model is a **narrow language helper**: short classification, extraction, routing, and templated drafting over **retrieved** context. It will **not** reason across documents, hold long context, do math, or write nuanced clinical narrative reliably — those stay deterministic SAS/R or escalate to a larger model. Everything reported comes from a validated engine (Pinnacle 21, Phoenix WinNonlin, the EDC/CTMS), and a human signs off. Used inside those lines, an on-device model is the most sovereign, cheapest, and most reproducible AI assist available.